Section 9: Multi-Turn Conversation Management

Refined Outline for Literature Review on LLM Context Management

J.C. Vaught

January 27, 2026

1 Introduction to Multi-Turn Conversation Management

Multi-turn conversation management represents a critical challenge in LLM deployment, where maintaining coherent, contextually-aware dialogue requires sophisticated strategies for context preservation, memory management, and state tracking. Recent research reveals that top open- and closed-weight LLMs exhibit significantly lower performance in multi-turn conversations than single-turn scenarios, with an average performance drop of 39% across six generation tasks, highlighting the urgency of effective conversation management techniques.

1.1 Core Challenges

- Performance degradation in extended conversations
- Context window limitations and computational constraints
- Token cost management in production systems
- Maintaining coherence across conversation turns
- Recovery from conversational errors

2 Truncation Strategies

Truncation strategies address the fundamental constraint of finite context windows while attempting to preserve conversational coherence and task-relevant information.

2.1 Sliding Window Approaches

2.1.1 Classical Sliding Window Attention

- Traditional sliding window techniques maintain fixed-size windows of recent conversation history
- Computational efficiency: reduces quadratic attention complexity
- Information loss: tokens outside attention window are ignored for prediction
- Practical limitations: models struggle to use information beyond approximately 1,500 words in practice

2.1.2 Recent Advances in Sliding Window Attention

- **SWAT (Sliding Window Attention Training)**: Replaces softmax with sigmoid function and utilizes balanced ALiBi and Rotary Position Embedding to address attention sink phenomenon and improve information compression
- **SWAA (Sliding Window Attention Adaptation)**: Plug-and-play toolkit combining five strategies: (1) applying SWA only during prefilling; (2) preserving sink tokens; (3) interleaving full attention/SWA layers; (4) chain-of-thought; (5) fine-tuning. Achieves 30-100% speedups for long-context inference with acceptable quality loss
- Hybrid approaches: Combining sliding windows with full attention layers shows promise for balancing efficiency and context retention

2.2 Selective Truncation

2.2.1 Importance-Based Selection

- Saliency detection for identifying critical conversation segments
- Entity-based retention strategies
- Task-relevance scoring for message selection

2.2.2 Observation Masking

- Hiding older, less important information from context
- Preserving high-signal information while reducing token count
- Application in multi-agent systems and long-horizon tasks

3 Conversation Summarization Approaches

Summarization techniques enable LLMs to maintain extended conversation history by compressing older segments while preserving semantic content.

3.1 Recursive Summarization

3.1.1 Core Methodology

- **Recursive Memory Generation**: Stimulating LLMs to memorize small dialogue contexts, then recursively producing new memory using previous memory and following contexts
- Generated summaries are significantly shorter than full dialogues, enabling handling of extremely long contexts across multiple dialogue sessions
- Progressive compression: Each summarization round condenses information while maintaining task-relevant details

3.1.2 Implementation Strategies

- Periodic summarization: Older conversation segments compressed at regular intervals
- Microsoft's approach: Append short, updated summaries before latest user message; remove older messages while retaining system prompt plus recent assistant/user pairs
- SLIM framework: Uses periodic summarization for context growth management in long-horizon agents

3.2 Mixture of Experts for Dialogue Summarization

3.2.1 Role-Oriented Approaches

- **MoE-based Summarization**: Role-oriented routing selects appropriate experts; fusion generation locates salient information to produce finalized summaries
- Specialized experts for different dialogue types and contexts
- Published at ACL 2024: "Dialogue Summarization with Mixture of Experts based on Large Language Models"

3.3 Context Compression Techniques

3.3.1 Dense Semantic Representations

- Compression ratios: 10:1 or better while maintaining response quality
- Token cost reduction: Smart memory systems achieve 80-90% token cost reduction
- Performance improvement: 26% improvement in response quality versus basic chat history management (Mem0 benchmarks)

3.3.2 Implicit Text Summaries for Retrieval

- LLM-based conversation summarizer for query and key generation
- Distilled lightweight conversation encoder to avoid extra inference costs
- Effective search enabling for few-shot dialogue state tracking

4 User Modeling and Personalization in Multi-Turn Dialogue

Personalization in conversational AI requires systems to build and maintain user models that evolve across conversation turns and sessions.

4.1 Dynamic User Profiling

4.1.1 Profile Construction Methods

- **Proactive Questioning**: Systems actively query users to construct detailed profiles
- **Memory Extraction**: Inferring user preferences and traits from conversation history

- **Implicit Metrics**: Session duration, response time, sentiment analysis, language complexity, typing speed
- **Explicit Feedback**: Direct user input on preferences and satisfaction

4.1.2 Recent Benchmarks and Datasets

- **PERSONAMEM**: Evaluates 15 state-of-the-art models on ability to internalize user traits, track evolving profiles, and generate personalized responses
- **PersonalLLM**: Published at ICLR 2025; provides prompts, responses, and personalities for testing personalization algorithms
- **PaRT (Personalized Real-Time Retrieval)**: Enhances proactive social chatbots with user profiling, intent-guided query refinement, and retrieval-augmented generation

4.2 Memory-Enhanced Personalization

4.2.1 Generative Approaches

- **Memory-Enhanced Conversational AI**: Employs distilled GPT-2 tokenizers for preprocessing; implements generative models with validation metrics including fluency, user satisfaction, memory recall, perplexity, diversity, and consistency
- Context-aware response generation based on historical interactions

4.2.2 Reinforcement Learning for Personalization

- **HumAIne-Chatbot**: Real-time personalized conversational AI via reinforcement learning
- Adaptive learning from user interactions
- Dynamic adjustment of conversation strategies

4.3 Privacy and Compliance

4.3.1 Regulatory Frameworks

- EDPB guidance (2024): AI memory must align with GDPR principles including lawful basis and data retention limits
- Design-level compliance requirements for enterprise deployments
- User control mechanisms: editing, deleting, and disabling memory features

5 Dialogue State Tracking

Dialogue state tracking (DST) maintains structured representations of conversation state, enabling task-oriented systems to understand user needs and execute appropriate actions.

5.1 Natural Language Dialogue State Tracking

5.1.1 NL-DST Framework

- **Interpretable and Robust DST**: Leverages LLMs to directly generate natural language descriptions of dialogue states, moving beyond traditional slot-value representations
- Enhanced state expressiveness and interpretability
- Evaluation on MultiWOZ 2.1 and Taskmaster-1 datasets

5.2 MultiWOZ Benchmark

5.2.1 Dataset Characteristics

- Multi-Domain Wizard-of-Oz dataset with fully-labeled human-human written conversations
- 10,000+ dialogues spanning multiple domains and topics
- Order of magnitude larger than previous task-oriented corpora
- Versions: MultiWOZ 2.2, 2.4 with annotation corrections for improved state tracking evaluation

5.2.2 Recent Extensions

- **Multi-User MultiWOZ**: Task-oriented dialogues between two users and one agent
- **ToolDial**: Published at ICLR 2025; multi-turn dialogue with tool integration

5.3 Error Detection and Action Generation

5.3.1 Reliability Improvements

- Approaches enabling reliable estimation of AI agent errors (January 2025)
- Guiding LLM decoders to generate more accurate actions
- State-of-the-art performance: CoDial on STAR dataset; UniPPN on MultiWOZ

6 Multi-Agent Shared Context

Multi-agent systems require sophisticated context sharing mechanisms to enable collaboration while managing memory and computational constraints.

6.1 Framework Architectures

6.1.1 AutoGen

- **Contextual Memory Model**: Each agent maintains short-term context through context_variables object
- Stores interaction history at agent level
- No built-in persistent memory
- Conversational collaboration philosophy

6.1.2 CrewAI

- **Layered Memory Architecture**: Built-in multi-tier memory system
- Short-term memory: ChromaDB vector store
- Recent task results: SQLite storage
- Long-term memory: Separate SQLite table based on task descriptions
- Entity memory: Vector embeddings for entity tracking
- Role-based organizational model

6.1.3 LangGraph

- **Dual Memory System**: In-thread memory for single tasks/conversations; cross-thread memory for data across sessions
- MemorySaver: Saves task flows linked to specific thread_id
- InMemoryStore or database backends for long-term storage
- Graph-based workflow approach

6.2 Enterprise Adoption Trends

6.2.1 Market Dynamics

- 72% of enterprise AI projects involve multi-agent architectures (2026), up from 23% in 2024
- Three dominant frameworks: LangGraph, CrewAI, AutoGen
- Distinct philosophies: role-based (CrewAI), graph-based (LangGraph), conversational (AutoGen)

7 Commercial Approaches to Conversation Memory

Major AI platforms have deployed diverse strategies for persistent conversation memory and context management in production systems.

7.1 ChatGPT Memory

7.1.1 Architecture and Approach

- **Automatic Context Inclusion**: OpenAI automatically includes details of previous conversations at the start of every conversation
- No explicit tool calls for memory access
- Persistent across sessions
- User controls: editing, deleting, disabling memory

7.2 Claude Memory

7.2.1 Implementation (September 2025 Release)

- **Visible Tool Calls**: Memory access implemented as transparent tool calls, providing visibility into when and how context is accessed
- Rolled out to all paid Claude users (October 2025)
- Remembers projects, context, and user-specified information

7.2.2 Project-Based Memory Architecture

- Separate memory banks for each project
- Ensures context separation (e.g., product launch planning separate from client work)
- Complete user visibility and editing capabilities
- Transforms Claude from stateless assistant to context-aware collaborator

7.2.3 Cross-Platform Compatibility

- Unique feature: Import saved memory data from other accounts or sessions
- Export/import from ChatGPT for context continuity
- Enterprise-focused design with privacy controls

7.3 Gemini Long Context

7.3.1 Context Window Capabilities

- **Gemini 1.5 Pro**: Introduced February 2024 with 1 million token context window
- Extended to 2 million tokens in production
- First model capable of accepting 1 million tokens then expanding to 2 million
- Research testing: Successfully tested up to 10 million tokens

7.3.2 Performance Characteristics

- Needle-in-a-Haystack (NIAH): 99% retrieval accuracy at 1 million tokens
- Near-perfect retrieval (¿99%) up to 10 million tokens in research settings
- Generational leap over Claude 3.0 (200k) and GPT-4 Turbo (128k)
- Capacity: 1 hour video, 11 hours audio, 30,000+ lines of code, or 700,000+ words

7.3.3 Industry Context

- Evolution from 4,000 tokens (ChatGPT launch, late 2022) to millions (2025)
- Industry standard: 128,000 tokens by 2025
- Cutting-edge: Llama 4's 10 million token capacity
- Gemini 1.5 Pro: Largest mainstream model context window as of late 2025/2026

7.4 Competitive Landscape

7.4.1 Cross-Platform Memory Features

- All major platforms deployed memory features by mid-2025: OpenAI ChatGPT, Anthropic Claude, Google Gemini, Microsoft Copilot
- Memory becoming baseline feature for enterprise and personal AI
- User control mechanisms across platforms: edit, delete, disable
- Balance between personalization and privacy

8 Session Management in Production Systems

Production deployment of conversational LLMs requires careful management of session state, context budgets, and computational resources.

8.1 Context Engineering for Production

8.1.1 Resource Constraints

- **Quadratic Cost Scaling**: Computational cost increases quadratically with context length due to transformer architecture
- Context window as scarce resource requiring optimization
- Token-based pricing: Longer contexts translate directly to higher costs
- For thousands of concurrent sessions, inefficient context utilization compounds costs exponentially

8.1.2 Context Engineering Principles

- Treat context window as limited attention budget
- Design retrieval, memory systems, tool integrations, and prompts around high-signal token maximization
- Keep context concise and relevant for both performance and cost efficiency

8.2 Context Window Budgeting Strategies

8.2.1 Token Allocation

- **Fixed Budget Approaches**: Allocate specific token counts to system prompts, user history, retrieval results, and generation space
- **Dynamic Budgeting**: Adjust allocations based on conversation phase and task requirements
- Practical ceiling on context size due to cost-performance tradeoffs

8.2.2 Monitoring and Optimization

- AI monitoring tools tracking token consumption patterns
- Identification of optimization opportunities
- Prevention of budget overruns
- Rate limit management: Requests Per Minute (RPM), Tokens Per Minute (TPM)

8.3 Context Management Approaches

8.3.1 Observation Masking vs. Summarization

- **Observation Masking**: Hide older, less important information; preserve high-signal context
- **LLM Summarization**: Generate short summaries of older conversation segments; theoretically enables infinite turn scaling without infinite context
- Hybrid approaches combining both techniques

8.3.2 Production Implementation Patterns

- Session state persistence across requests
- Context window utilization monitoring
- Graceful degradation when approaching limits
- User feedback integration for context relevance

9 Performance Challenges and Recovery Strategies

Recent research has identified significant performance degradation in multi-turn scenarios and proposed strategies for mitigation.

9.1 Performance Degradation Patterns

9.1.1 Empirical Findings

- **39% Average Performance Drop**: Top open- and closed-weight LLMs show significantly lower performance in multi-turn vs. single-turn conversations across six generation tasks
- Minor loss in aptitude combined with significant increase in unreliability
- LLMs make premature assumptions in early turns, attempting to generate final solutions too quickly
- **Lost in Conversation**: When LLMs take wrong turns, they fail to recover

9.2 Mitigation Strategies

9.2.1 Fine-Tuning for Multi-Turn Performance

- Critical considerations: How to handle loss computation during training
- Research suggests traditional loss masking may not be optimal
- Action-Based Contrastive Self-Training for clarification in multi-turn contexts

9.2.2 Conversation Planning

- **SCOPE (Semantic space Conversation Planning with improved Efficiency)**: Exploits dense semantic representation of conversations
- 70x faster than conventional simulation-based planning algorithms
- Enables broader conversation search and improved turn-level decisions

10 Future Directions and Open Problems

10.1 Emerging Challenges

- Scaling to extremely long conversations (100,000+ turns)
- Maintaining consistency across extended dialogue
- Efficient memory consolidation and retrieval
- Balancing personalization with privacy preservation
- Multi-modal conversation state tracking

10.2 Research Opportunities

- Hierarchical context management across multiple timescales
- Neural-symbolic hybrid approaches to dialogue state
- Cross-session learning and transfer

- Adaptive context budgeting based on task complexity
- Improved error detection and self-correction mechanisms

Key Takeaways

1. Multi-turn conversation management remains a critical bottleneck for LLM applications, with significant performance degradation (39% average) compared to single-turn scenarios
2. Modern truncation strategies have evolved beyond simple sliding windows to include hybrid approaches (SWAT, SWAA) that balance efficiency with context retention through techniques like sink token preservation and layer interleaving
3. Summarization approaches, particularly recursive summarization and mixture-of-experts methods, enable 80-90% token cost reduction while maintaining or improving response quality
4. User modeling and personalization have transitioned from research concepts to production features, with all major platforms (ChatGPT, Claude, Gemini) deploying memory systems by mid-2025
5. Commercial approaches show diverse strategies: ChatGPT's automatic inclusion, Claude's transparent tool-based access with project separation, and Gemini's massive context windows (2 million tokens in production, 10 million in research)
6. Multi-agent frameworks (AutoGen, CrewAI, LangGraph) have achieved 72% enterprise adoption by 2026, each offering distinct memory architectures suited to different collaboration patterns
7. Context engineering has emerged as a critical discipline for production systems, treating the context window as a scarce resource requiring careful budgeting, monitoring, and optimization
8. Dialogue state tracking has evolved to natural language representations, moving beyond slot-value pairs to enable more expressive and interpretable state management
9. Future challenges center on scaling to extremely long conversations, maintaining consistency, and balancing personalization with privacy while managing computational costs